AREAS 2

AREA KULLANILAN YERLER 2

AREA OLUŞTURMA 3

AREA ATTRİBUTE'U 4

AREA'YA ROUTE BELİRLEME 5

AREA'YA ÖZEL ROUTE TANIMLAMA(MAPAREACONTROLLERROUTE) 6

AREA'LAR ARASI BAĞLANTI OLUŞTURMA 7

AREA'LAR ARASI VERİ TAŞIMA 7

Areas

Area, bir web uygulamasında farklı işlevsellikleri ayırmak için kullanılan bir özelliktir. MVC'de kullanılıyor genellikle.

Standart bir web uygulamasında, bir normal kullanıcıların erişebildiği sayfalar vardır bir de yetkililerin ulaşabildiği admin panel benzeri yönetim sayfaları vardır.

Bu şekilde geliştirilen uygulamalar için Asp.Net Core mimarisinde bu katmanlar Area olarak geliştirilebilir, katmanlanabilir.

Tabi ki zorunlu değil bir tercihdir.

Kullancının gördüğü sayfalar farklı işlevselliklere sahipken, yetkililerin gördüğü sayfalar farklı işlevselliklere sahip. Örneğin bir e ticaret uygulamasında bir kullanıcı sadece ilanları görebilirken, bir yönetici ilanları ekleyip kaldırabilir.

Her Area içerisinde View, Controller ve Model barındırabilir.

Bu sayede Asp.Net Core uygulamarında yönetilebilir, küçük ve işlevsel gruplar oluşturabilir.

Area Kullanılan Yerler

- Yönetim Paneli
- Faturalandırma Sayfaları
- İstatistiksel Paneller
- Mantıksal olarak ayrılabilen üst düzey işlevsel bileşenler
- Vs.

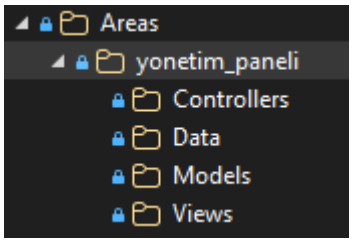
Tabi yer şeyde de Area kullanılmaz mantıksal olarak ayrılabilen olması lazım. Bokunu çıkarmamak gerek. Area çok katmanlı mimari değildir. Area'lar sadece Presentation katmanında bir klasörde barındırılan ve kullanılan yapılardır. Aslında Area'lar düzene sokar Controller yapılarını vs.

Area Oluşturma

Area oluşturmak için proje altında “Areas” isimli bir klasör oluşturulur. Bu isim özel bir isimdir. Sadece ismi “Areas” olan klasöre sağ tıklayınca Area ekleme seçeneği çıkar. Aynı sadece Controllers klasöründe Controller seçeneği çıkması gibi.

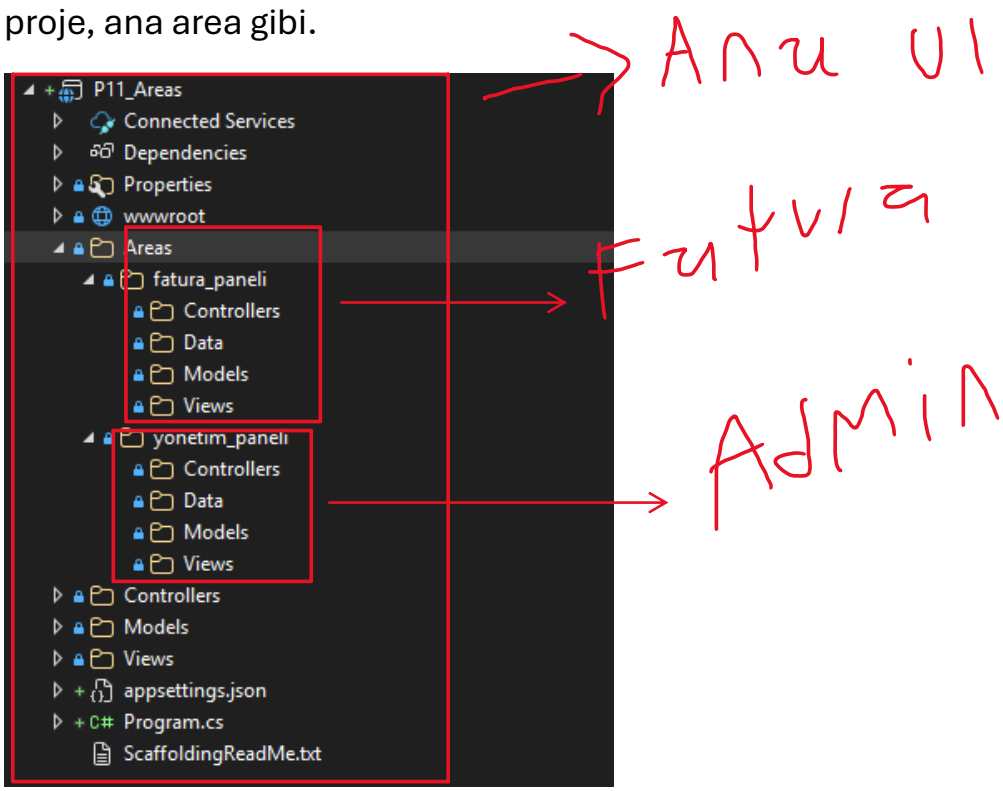
İlgili klasöre sağ tıklayıp Area seçeneğini seçilir ve çıkan menüde MVC Area seçeneğine basılır. Burada Area ismi çok önemli değil uygun bir şey verilir.

İlgili Area klasörünün altında belirtilen isimde bir klasör oluşturulur. Bu klasör içerisinde ise ilgili Area’nın Controller, Model, Data, View gibi klasörleri oluşur.



Sanki Local bir MVC uygulaması gibi.

Bu uygulamada, ana proje kullanıcının göreceği alan Area’sı. Yonetim_paneli Area’sı ise Admin Panel kısmı gibi düşünülebilir. Yani ana proje, ana area gibi.



Area Attribute'u

Area'lar için Controller eklendiği zaman, ilgili Controller Area Attribute'u ile etiketlenmelidir. Çünkü mesela hem orijinal projede hem de yönetim_paneli Area'sında bir Home Controller olsun. Bu durumda mimari hangi Controller'a gideceğini bilemez ve hata verir. Farklı Area'lardaki Controller'ların çakışmaması için bu işlem yapılır.

İlla aynı isimde olmasına gerek yok genel olarak tüm Area Controller'ları Area Attribute'u ile işaretlenmesi gerekiyor.

```
[Area("yonetimPaneli")] // Area Attribute'u
0 references
public class HomeController : Controller
{
    0 references
    public IActionResult Index()
    {
        return View();
    }
}
```

Bu kullanımdan sonra mimariye, bu Area'ya gelen isteklerde Home Controller'ı tetikle, aski halde ana Home Controller'u tetikle denmiş olur.

Burada kullanulan isim ile Area ismi aynı olmak zorunda değil. Mesela projede Area ismi "yonetim_paneli" iken Area Attribute'u "yonetimPaneli". Fakat kullanımı rahat olsun diye genellikle Area ile aynı isimlendirme kullanılır.

Bu tanımlamlardan sonra Area ismi belirtilmediği sürece ilgili Area'nın Controller'ları tetiklenmez.

Area’ya Route Belirleme

Her bir Area içerisindeki Controller’lara erişim için özel bir Route’a ihtiyaç duyar.

Program.cs içerisinde normal Route tanımlar gibi tanımlanabilir:

```
app.MapControllerRoute(
    name: "areaDefault",
    pattern: "{area:exists}/{controller=Home}/{action=Index}/{id?}"
    );
```

Burada aynı normal bir Route gibi bir name alır Route. “{area}” aynı controller veya action gibi ön tanımlı özel bir parametredir. Mimari bu parametreyi tanır. Area geldikten sonra controller ve action bilgileri geliyorsa bu Route çalışacak.

Normal Route’dan farklı olarak Area paramteresine exists Constrait’i uygulanır. Yani Area mevcut ise eğer demektir bu aslında.

Bu kullanım default Area Routa tanımlamak için kullanılır. Yani bu tanımlamda Area’ların çalışması için default/gerekli bir tanımlamadır.

Artık “yonetim_paneli/home/Index” Url’sine gidince yonetim_paneli Area’sı altındaki Home Controlle ve Index Action’ı tetiklenir.

Burada yonetim_paneli ismi Area’nın isminden değil Area Atrribute’tuna verilen isimden dolayı öyle yazılır. Yani Area Attribute’unda mahmut yazsaydı, “mahmut/home/Index” Url’sine gidince yonetim_paneli Area’sının elemanları tetiklenirdi.

Area’ya Özel Route Tanımlama(MapAreaControllerRoute)

MapControllerRoute default Area rotası belirlemeye yarar.

MapAreaControllerRoute ise bir Area’ya özel rota belirlemeye yarar.

Bu fonksiyon MapControllerRoute’dan farklı olarak “areaName” isimli bir parametre daha alır.

```
app.MapAreaControllerRoute(
    name: "areaRoute1",
    areaName: "yonetim",
    pattern: "yonetim_sayfasi/{controller=Home}/{action=Index}/{id?}"
    );
```

Şu anda “yonetim” Attribute’u ile işaretlenmiş tüm Controller’lar “yonetim_sayfasi” ile başlayan rota ile tetiklenir. Yani

yonetim_sayfasi/ Controller / Action şeklinde gelen tüm Url’ler “yonetim” Attribute’u ile işaretlenmiş Controller’ları tetikler. Buradaki “yonetim_sayfasi” önemli değil mahmut da yazılabilir buraya bir keyword gibi bir şey bu. Önemli olan yer areaName.

```
app.MapAreaControllerRoute(
    name: "areaRoute2",
    areaName: "fatura_paneli",
    pattern: "fatura/{controller=Home}/{action=Index}"
    );
```

Bu da fatura paneli için özel bir rota.

Area’lar Arası Bağlantı Oluşturma

Area’ları birbirine bağlarken, rota içerisinde Area parametresini de göndermek gerekir. Bu işlem HtmlHelpers ile veya TagHelpers ile yapılabilir.

HtmlHelper ile:

```
@Html.ActionLink("Yönetim", "Index", "Home", new {area = "yonetim"})
<a href="@Url.Action("Index", "Controller", new { area = "yonetim"})"></a>
```

TagHelper ile:

```
<a asp-area="yonetim" asp-controller="Home" asp-action="Index">Yönetim</a>
```

TagHelper’da direkt asp-area attribute’u var. Bu sebeple TagHelper her zamanki gibi daha iyi.

Area’lar Arası Veri Taşıma

Bir Areadan diğer Area’ya veri taşımak için Tempdata kullanılabiliyor.

Tempdata Action’lar arası veri taşımak için kullanılıyor normalde. Bu sayede herhangi bir Area altındaki Controller altındaki Action’daki bir değeri, başka bir Area altındaki Controller altındaki Action’a taşıyabilir.

Veri taşımak için gönderilecek Action’dan gidilecek olan Action’a Redirect etmek gerekir.

RedirectToAction fonksiyonunu kullanırken. İlk ve ikinci parametre her zamanki gibi Action ve Controller değerli oluyor. Fakat 3. Parametre olarak Area belirtilmesi gerekir. Eğer Area belirtilmez ise anlık olarak içerisinde bulunan Area’da verilen Controller ve Action’ı bulmaya çalışır. Buradaki Area ismi Attribute’ta verilen değil gerçekten Area’nın ismidir.

Gönderen:

```
public IActionResult AreaArasiVeriTasima()
{
    TempData["veri"] = "Fatura Panelinden gelen veri";
    return RedirectToAction("Index", "Home", new {area = "fatura_yonetimi"});
}
```

Alan:

```
public IActionResult AreaArasiVeriTasima()
{
    var data = TempData["data"];
    return View();
}
```